Advanced Machine Learning
Reinforcement learning

Master MLDM

Practical Assignment

Objective of this practical session:

- Study reinforcement learning algorithms on a simple project.

You should write a report explaining your work and your results with relevant discussions. Take to write precisely your experimental setting. You have also to provide your code.
Deadline: Wednesday October 8, 2025. To be uploaded as a compressed archive on Moodle in the *Bandits and Reinforcement Learning* section of the *Advanced Machine Learning* course workspace.

1 Reinforcement Learning

We consider here a simplified version of the PAC-MAN video game: PAC-MAN lives in a maze discretized in $N \times M$ squared and can move from one square to an adjacent one using for actions: UP, LEFT, RIGHT, DOWN. The maze contains some blocks that PAC-MAN cannot pass through, a square with a PAC-DOT that represents the goal of the game and a block with a GHOST, the other blocks being empty. If PAC-MAN reaches the GHOST, the game is over. PAC-MAN receives a reward of $+10$ if it reaches the PAC-DOT, -10 if it reaches the GHOST square, -1 point for the other blocks. PAC-MAN can starts at any position except at PAC-DOT and GHOST squares.

```
 - - - - -
|.|.|.|.|P|
 - - - - -
|.|.|B|B|B|
 - - - - -
|G|.|B|.|.|
 - - - - -
|.|.|D|.|.|
 - - - - -
```

Table 1: An illustration of the game on a 4×5 blocks game: `P` represents the position of PAC-MAN, `B` the blocks, `G` the ghost `D` the PAC-DOT, `.` defines an empty block.

The idea is to learn a policy for maximizing the sum of rewards received by the agent PAC-MAN.

- Implement an environment defining the game to play - you can use a text interface for representing the game or a GUI if you have time and are interested by this feature[1]. You can assume that the definition of the game is fixed and the agent plays always with the same configuration, *i.e.* the room remains unchanged between 2 games.

[1]If you prefer, you can choose to use the Gym library and select one (equivalent) game from this library: https://www.gymlibrary.dev. This may require more installation and development time. There are actually a lot of environments for PAC-MAN games.

- Implement a learning algorithm based on Q-learning for solving this problem. Define a setup for evaluating the behavior of the agent and the quality of the algorithm (do no forget to test different parameters). We expect here that PAC-MAN will learn to go directly to the PAC-DOT, avoiding the ghost and block square.
- Implement a learning algorithm based on a Monte-Carlo Tree Search. How does it scale in comparison to the previous algorithm?
- Introduce some randomness in the agent moves: PAC-MAN may move in the wrong direction and move randomly in a direction different from the policy with (small) probability. In this case, even if you learned a perfect policy, the agent may fail due to the randomness of the procedure. What is the behavior of the agent in this context?

Optional but advised We now would like to study the interest of using a neural net to predict the action given a state. Try to propose a way to use and train this neural net for learning the policy, we assume again that the blocks of the room remain unchanged between two different games. Again do not forget to evaluate your solution with respect to different parameters, and test when some randomness is added to the moves of the agent.

Optional Now, we assume that the configuration of the room changes (randomly) between 2 consecutive games: a.k.a. the ghost can move to another adjacent square. Adapt the solution(s) proposed previously to learn from changing configurations. Again, do not forget to consider different parameters and test when some randomness is added to the agent moves.

Optional Propose your own extension! (add more PAC-DOT and/or more ghosts for example)